EasyMeals

By: Dean Leon, Ayowade Owojori, Dylan Peniuk, Noam Yaffe

Team info

Roles

- Dean Leon - Full-stack developer → Has experience on both sides of the development stack and can help with tasks on all ends
- Ayowade Owojori - Backend lead → Has participated in a lot of projects surrounding backend development, and feels comfortable with the tools that we're using
- Dylan Peniuk - Project leader → Came up with the idea, and has the most concrete view on what the project should do, what it should look like, etc.
- Noam Yaffe - Frontend lead → Most experience with UI/frontend design and implementation

GitHub Repository

- https://github.com/yaffenator/EasyMeals

Communication Channels

- Microsoft Teams

Product description

Abstract

Meal prepping is hard. For any individual receiving food benefits or other financial aid, the idea of properly allocating a portion of their monthly income to groceries as effectively and efficiently as possible may seem like a daunting task. Introducing: EasyMeals – Automate your meal-prepping goals with a personalized AI nutrition expert. This website will allow you to enter the amount of money you allocate to food and groceries each month and select your physical goals (such as losing weight or building muscle) and their extent. Using this information, EasyMeals will create a month-long meal-prepping plan, showing you what groceries to buy, how much of each grocery item you will need, and some fun meals that you can prepare to satisfy your hunger.

Goal

The goal of this project is to make a website that allows people to skip the trouble of budgeting for food and the stress of finding recipes to prepare, by having it all in one place. It will allow people to enter their monthly food budget and automatically give a list of groceries to buy and recipes of what to cook week by week. It will also ask for food allergies and restrictions to let

people be stress-free about what they are eating. Overall, this project would reduce the stress of wondering what you are able to afford and what you are able to make, by laying it all out month by month.

Current practice

Today, this is similarly found in meal plans like Hello Fresh, where you pay a monthly subscription, and they deliver ingredients to your house. The problem with this is that it is expensive, and it's impossible to know where that food is really coming from. Also, for lower-income families who rely on government aid programs like EBT/food stamps, they are unable to use that money for those programs.

Novelty

What makes our approach different is the introduction of an AI assistant that will webscrape products from local grocery stores to create specialized meal plans that are applicable to user requests. This approach will factor in the price of the ingredients, which is absent from existing services, and the ability to cook the meals yourself, where services like Factor_ ship the entire meal.

Effects

Everyone should care about their expenses with food. However, this is targeted more towards people with lower incomes, or multiple jobs, who worry about having enough money for food or spending the time to research what to make. This would make a difference because people would have easy access to find meals that work for their budget and schedule.

Use Cases

1. (Noam Yaffe) The user will narrow down their fitness goals through a variety of health-related inquiries.
    a. Actor: Registered user
    b. Triggers: Upon registering for the website, the user is immediately met with health/fitness-specific questions
    c. Preconditions:
        i. The user is logged into the system.
        ii. The user knows the state of their physical well-being, whether that is their body weight, weekly activity level, etc.
    d. Postconditions:

i. The user has properly responded to all health & fitness related questions.
ii. The system will use this information to fit the generated meal plan to the user's health and fitness goals.

e. Success Scenario:
   i. The user successfully answers all questions relating to their fitness & health goals
   ii. The system fine-tunes the generated meal-prepping plan based on the user's health & fitness inputs.

f. Extensions &Variations:
   i. Weight loss vs weight gain: depending on the user's goals, they could additionally list the amount of weight that they hope to gain or lose within the specified one-month timeframe.
   ii. Activity level descriptions: if time allows, we can additionally ask the user to describe their weekly activity level through the activities they participate in, which would give our model further insight into their current physical activity and needs.

g. Exceptions:
   i. Invalid inputs: The user enters information that the system cannot process
   ii. Faulty API call: Each user receives a designated API request to our AI model, which generates a meal-plan tailored to their needs. However, whether this works depends on the API key functioning properly alongside other factors.

2. (Dylan Peniuk) The system identifies SNAP-eligible items within a generated grocery list and tracks the remaining monthly benefit balance to ensure the user never exceeds their allotted aid.
   a. Actor: Registered user
   b. Triggers: The user generates an ingredient list
   c. Preconditions:
      i. The user has entered their monthly SNAP benefit in their profile
      ii. A meal and grocery list has been created
   d. Postconditions:
      i. The grocery list is visually partitioned into "EBT Eligible" and "Cash Required" sections.
      ii. The system confirms the total eligible amount is less than the user's remaining SNAP balance.
      iii. The user is notified of any "out-of-pocket" costs
   e. Success Scenario:
      i. Activate Filter: The user enables "EBT Optimization" in their plan settings.

ii. Item Validation: The system cross-references the generated grocery list against the SNAP-eligible food list
iii. Balance Calculation: The system subtracts the estimated cost of eligible items from the user's current "Benefit Balance."
iv. Item Tagging: The system generates the final list, marking eligible items with a specific icon
v. Out-of-pocket summary: The system provides a secondary total for items that require cash or debit
vi. Confirmation: The user reviews the breakdown and saves the list

f. Extensions &Variations:
i. Combined Payment View: For users who use both SNAP and cash, the system provides a "Split-Payment" preview so they know exactly how much will be charged to each card at the register.

g. Exceptions:
i. Benefit depletion: The generated plan exceeds the monthly SNAP balance
ii. Database sync error: The system cannot verify the SNAP status of a new or niche product

3. (Dean Leon) The user will receive an itemized grocery list and curated recipes tailored specifically to their financial constraints and physical objectives.
a. Actor: Registered user
b. Triggers: The user generating the plan (i.e. clicking a button)
c. Preconditions:
i. User is logged in
ii. User has completed their health profile
iii. Grocery price data is available
d. Postconditions:
i. The user receives a downloadable/viewable grocery list.
ii. The user receives a curated meal plan where the total cost is <= User Budget.
iii. The total nutritional output matches the user's physical objectives.
e. Success Scenario (Main Success Path):
i. Input Parameters: User enters their maximum budget and goal.
ii. System Calculation: The system filters recipes that fit the nutritional macro-profile.
iii. Cost Optimization: The system cross-references ingredients with current market prices to find the most cost-effective combination.
iv. Review: The system presents the proposed meal plan and total estimated cost to the user.
v. Finalization: The user accepts the plan.

vi. Delivery: The system generates an itemized grocery list organized by store aisle and provides step-by-step cooking instructions for recipes.

f. Extensions & Variations:
   i. Dietary Restrictions: The user can toggle filters for vegan, gluten-free, or keto-friendly options within the same budget.
   ii. Inventory Integration: The user can "check off" items they already have at home (e.g., salt, oil, rice) to further reduce the total cost.

g. Exceptions (Failure Scenarios):
   i. Insufficient Budget: The user's budget is too low to meet their caloric/nutritional needs
   ii. API Timeout: Real-time grocery pricing data is unavailable.
   iii. Conflicting health goals

4. (Ayowade Owojori) The website will prioritize grocery stores within the user's physical proximity, and use them as a baseline for item pricing & meal-prepping.
   a. Actor: Registered user
   b. Triggers: The user inputs (or updates) their zip code location
   c. Preconditions:
      i. The user has entered (or updated) their location via zip code
      ii. The user completed the fitness and health goals questionnaire
   d. Postconditions:
      i. The system finds grocery stores in proximity to the user's inputted zip code, and uses them to estimate the costs of certain food items
   e. Success Scenario:
      i. The system creates a meal-prepping plan that also accounts for the user's proximity, alongside their budget constraints and fitness goals
   f. Extensions and Variations
      i. The system can use stores in proximity to the user to compare different food items' prices, therefore generating the most cost-effective, location-based meal-prepping plan for the user
   g. Exceptions
      i. Location denied: The user refuses to share location or zip code
      ii. No supported retailers: The user is in an area where food prices can be determined online
      iii. Stale data: The local price feed hasn't been updated in over 24 hours.
      iv. The user is outside the supported region of the program

Non-functional Requirements

- Scalability: The system will be able to handle 100 meal-prep generation requests per day.

- Usability: The system should generate the grocery list and the meal plan in one minute.
- Security: All user data will be kept confidential, ensuring that each user is completely comfortable sharing information about their health habits and goals.

External Requirements (from Project Requirements Elicitation)

- The product must be robust against errors that can reasonably be expected to occur, such as invalid user input.
- The product must be installable by a user, or if the product is a web-based service, the server must have a public URL that others can use to access it. If the product is a stand-alone application, you are expected to provide a reasonable means for others to easily download, install, and run it.
- The software (all parts, including clients and servers) should be buildable from source by others. If your project is a web-based server, you will need to provide instructions for someone else to set up a new server. Your system should be well-documented to enable new developers to make enhancements.
- The scope of the project must match the resources (number of team members) assigned.

Technical approach

To build this meal-prepping website, we will use the React library combined with a Next.js framework. We chose this tech-stack as it will allow us to leverage features like automatic code splitting, server-side rendering, and more. We plan to use TypeScript (+ TailwindCSS as a replacement for HTML/CSS) for our frontend components and Python for our backend functions. We will send and receive information from our fine-tuned AI model, which will most likely be implemented via a Gemini API handler, and MongoDB (or Firebase, Supabase, etc.) as our database to store individual user information and preferences. If time permits, we also plan to implement a user authentication system using both basic email/password authentication and more advanced OAuth (such as Google and other OAuth providers).

Risks

The biggest risk of this project is making sure that we find the prices of food accurately, so we don’t put people over budget. Another risk is making sure we fully exclude all items that are flagged as food restrictions as not to be liable for an allergic reaction or other response to food the user shouldn’t be eating. Another risk is that our program will collect sensitive user data (weight, lifestyle, ect.), so our program should be secure enough to ensure that this data is kept private.

Additionally, add the following:

- 4+ major features you will implement.
  - A weekly grocery list that stays within the user's budget
  - A full month of recipes for meals the user can cook at home
  - Meals targeted towards fitness goals
  - Allowing the user to add food restrictions to prevent allergic reactions
- 2+ stretch goals you hope to implement.
  - Allow the user to add family members to their meal plan (prep for a family of 4)
  - Allow a certain selection of grocery stores to purchase from nearby

The major features should constitute a minimal viable product (MVP).

Timeline

Week 3:

- Wade and Dean
  - Set up the codebase and choose tech stack (most likely React + Next.JS, TypeScript, etc.)
- Noam & Dylan
  - Outline UI components & different pages in the web application

Week 4

- Noam and Dylan
  - Implement UI and pages in the project
- Wade and Dean
  - Make backend for user inputs for specifying meal plan requirements

Week 5

- Wade and Dean
  - Implement the Gemini API
  - Begin testing meal plan generation
- Noam and Dylan
  - Finalize frontend design, use case 1 design complete (implemented on local website)

Week 6

- All members
  - Present progress and idea and midterm project presentation
  - Test grocery list and meal plan generation and fix bugs such as the generated content including items or meals the user has in their restriction settings.

Week 7

- Wade
    - Begin end-to-end (E2E) integration and ensure data flows smoothly from user input to the final displayed meal plan

Week 8

- All members
    - Begin system testing → includes backend function tests, progress coordination with the entire team, and work combination/merging.
- This is the point where **external feedback** will be very useful to the team and the project's overall progression.

Week 9

- All members
    - Finalize system testing and ensure that the base product is functional.
    - Potentially move on to implementing some of our listed stretch goals.

Week 10

- All members
    - Finalize the project and set up the final project presentation.

Software architecture

Chosen software architecture: **Layered architecture pattern**

Provide an overview of your system. Specifically:

- Identify and describe the major software **components** and their functionality at a conceptual level.
    - User-Facing Frontend: Built with React and Next.js, this specific component will handle the user interface, visualizing all user interactions within our website.
    - Application Server: A Python-based backend that acts as the brain of the program. Takes the user-inputted data regarding their health & fitness, constructs optimized prompts to send to our Gemini API, and receives the API response to be refactored and displayed to the user.
    - Gemini API: Receives the data from the business logic and returns a formatted JSON object containing the month-long meal plan and grocery list.

    - Database Backend: A series of collections stored in a Firestore database, containing instances and attributes that correspond to different user profiles, meal plans, and saved grocery lists.

- Specify the **interfaces** between components.
    - Frontend ↔ Backend: The frontend sends JSON payloads of user preferences, and the backend returns structured meal data.
    - Backend ↔ Gemini API: Utilizes Google's Gen-ai SDK to send prompts and receive structured text and JSON responses.
    - Backend ↔ Database: Uses Firebase to perform CRUD operations on user data.

- Describe in detail what **data** your system stores, and how. If it uses a database, give the high-level database schema. If not, describe how you are storing the data and its organization.
    - To store our data, we will connect our web application to a Firestore database (Firebase). This development platform provides a NoSQL, Schema-less database for us to store user information, meal plans, and other important data. That said, since the database does not directly use a Schema to operate, we will instead provide a high-level description of which collections will store what attributes, and the specific data to be stored inside of them.

- If there are particular **assumptions** underpinning your chosen architecture, identify and describe them.
    - Assumes a persistent internet connection for Firebase and Gemini uptime.
    - Assumes grocery prices that are scraped are updated at least every 24 hours.
    - Assumes user is located in the US.
    - Assumes that the user has an idea of their current health status and their future meal-planning & fitness goals.
    - Assumes that the user knows the specific amount of monthly benefits that they receive for personal food security.

- **For each of two decisions pertaining to your software architecture**, identify and briefly describe an **alternative**. For each of the two alternatives, discuss its pros and cons compared to your choice.
    - Alternative 1: Microservices Architecture
        - Splits the AI logic, user management, and grocery scraping into separate deployable services.

        - Pros: Highly scalable, if one aspect crashes or goes down, the rest of the application will stay up
        - Cons: Overly complex for a team of 4. High overhead in managing inter-service communication and deployment.
    - Alternative 2: Client-server architecture
        - Runs all logic, including AI calls and data processing from the React frontend, using Firebase as a direct backend service.
        - Pros: Faster, no need to maintain a Python backend
        - Cons: Significant security risks in exposing API keys, poor performance on mobile devices, and limited web scraping capabilities.

Software design

Provide a detailed definition of each of the software components you identified above.

- What packages, classes, or other units of abstraction form these components?
    - Frontend (Next.js/React):
        - Components/
        - Context/
    - Backend (Python/Flask or FastAPI):
        - Scraper/
        - AI_Engine/
    - Database (Firebase):
        - Firestore
        - Firebase Auth
- What are the responsibilities of each of those parts of a component?
    - Components/ will hold reusable UI elements
    - Context/ will hold global user states
    - Scraper/ will hold modules dedicated to finding princes and information from local retailers
    - AI_Engine/ will hold the logic for crafting and fetching the prompts from the Gemini API
    - Firebase will be realtime storage
    - Firebase Auth will hold OAuth and email/password tokens

Coding guideline:

Coding guidelines for the programming languages, frameworks, and platforms we will use:

- TypeScript: Follow coding conventions from TypeScript's official documentation: https://www.typescriptlang.org/docs/

- Python: Follow PEP 8 coding standards: https://peps.python.org/pep-0008/
- React: Follow the library's strengths & utilization conventions as provided in the documentation: https://react.dev/
- Firebase: The development platform has a lot of documentation surrounding its database and provides authentication systems: https://firebase.google.com/

Process Description:

Risk assessment:

Gemini wrapper is generating too many responses

- Likelihood: Low
- Impact: High
- We all know that this is a possibility so we will all be careful to avoid it which is why the risk is low. The impact would be high though because it could be expensive if it goes wrong.
- We are taking steps to avoid this by putting limiters on our tests and responses so we can exceed the response limit.
- We would have it output to a terminal how many tokens the response used and how many are left to use, to be additionally sure we aren't going to hit the token limit.
- If we were to exceed the limit and charge more than what is willing to be spent we would stop using it immediately and wait for the token usage to reset before continuing.

AI-generated recipe including ingredients the user is allergic to

- Likelihood: Low
- Impact: High
- It's low likelihood because we will make sure that is included in the prompt so gemini will know not to use that in a recipe. The impact is high because the user isn't able to use a recipe if they are allergic to it or the user won't notice and could have an allergic reaction.
- We will have strict guidelines for gemini to follow so that it ensures that it doesn't include recipes with allergens.
- For detecting the problem, we could make gemini look back through to ensure no allergens were included. If they were, we would add something in our backend that highlights the text red or just deletes the recipe entirely so there are two forms of checking.
- Should it occur, we will look at other ways of mitigating the error.

Leaked API Key for Gemini

- Likelihood: Medium
- Impact: High
- API keys can accidentally be published to version control and web scrapers work to detect accidentally published API keys on GitHub within seconds of a publish. A leaked key can cause major financial damage as attackers can rack up API requests.
- To mitigate: We will restrict the key so it only accepts prompts from specific production IP addresses and our Firebase server IP. Prompts to Gemini will not contain information pertaining to a specific user, so that user data is not stored in any model logs. Use .gitignore to hide our API key during development and use “git add .” instead of “git add *” to avoid pushing sensitive environment variables.

Overwriting each other’s features

- Likelihood: High
- Impact: Low
- These are accuracy because it happened a lot last term in our SWE 1 class but we always got it fixed
- We will make sure only one person is working on a file at a time to prevent these errors from happening.
- This problem is auto detected by github when pushing or pulling a file with merge conflicts
- If this happens the people who worked on the file will meet to revise the conflict

Firebase security breach leaks user information

- Likelihood: Low
- Impact: High
- Firebase is built on Google Cloud Platform’s infrastructure, which adheres to global security standards (SOC 1/2/3, ISO 27001). The primary point of failure is typically not the provider, but poorly configured security rules by developers.
- We will mitigate this by testing and peer-reviewing our code.

Project schedule

Milestone	Tasks	Prerequisites
User Interface	-Build a cohesive, professional landing page. -Landing page includes the purpose of the project, how the project works, etc. -Create a section for users to sign up or log into their EasyMeals account	-Set up codebase (DONE) -Choose frontend tools, frameworks, and libraries (DONE)
Authentication System	-Create a working, user/password-based authentication system for users to create an account or log into a preexisting one -If time permits, add an option to implement Google OAuth.	-An authentication system has been chosen for the website's auth to be implemented with. -UI has been created for the authentication page of the website.
Health & Fitness Questionnaire	-Create a UI for users to enter information about their health & fitness status and goals. -Store information in the database.	-Choose a database to store user-specific information (DONE) -Connect the working database to the project website.
User Dashboard	-Generate a monthly, week-by-week meal-prepping plan specific to the user's answers in the questionnaire. -Display this meal-prepping plan through a coherent, professional-looking UI.	-UI & Health Questionnaire. -Working database connected to the project website.
Gemini API	-Generate API Key -Choose model -Share API key with members and set up environment variables on all development machines	-Python environment configured.
Core AI Logic	Develop backend prompt engineering to convert user data into a formatted JSON object	-Gemini API & Questionnaire data.

Grocery and Recipe Engine	Implement grocery list generation and recipe formatting; integrate basic web-scraping for pricing.	-Core AI Logic.

Team structure:

- Dean Leon - Full-stack developer → Has experience on both sides of the development stack and can help with tasks on all ends
- Ayowade Owojori - Backend lead → Has participated in a lot of projects surrounding backend development, and feels comfortable with the tools that we're using
- Dylan Peniuk - Project leader → Came up with the idea, and has the most concrete view on what the project should do, what it should look like, etc.
- Noam Yaffe - Frontend lead → Most experience with UI/frontend design and implementation

Test plan & bugs

- We want to do E2E tests for the following using either Playwright or Vercel's agent-browser:
    - Authentication
        - Testing that the user login, signup, and signout work without errors
        - We want to test this because if this breaks, users lose control over their accounts, and that would be a crucial bug.
    - Meal-plan generation
        - Testing that the new changes that are made don't break the AI generation of the meal plans.
        - This is the main attraction of our website; if this is not working, our website is of no use, therefore, we need to always ensure our generation works.

Documentation plan

- We hope that the user can navigate through the system without any documentation or guide on how to use the website. As for developer guides, we will include a docs folder with instructional markdown files. For example, we'll have a quick start markdown file that will have steps for starting up the website locally. Additionally, for any important

setup steps for the Firebase database or the Gemini API setup we will create markdown files with instructions.